COMUNICACIONES II – 27145

PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Quezada Gómez Juan David - 2204650
Guerrero Cruz Daniel Ricardo - 2200535
Diaz Rodriguez Edwar Alexis - 2194682

https://github.com/DanielRGC/commII_A1_G4

1. ABSTRACT

This practice aims to provide the knowledge and skills necessary to understand and program software-defined radios (SDR) using GNURadio. It covers both theoretical concepts and their practical application, including the study of basic SDR principles, the implementation of blocks in GNURadio, and the analysis of the obtained results. For this purpose, software such as Ubuntu Linux, GNURadio, and GNURadio Companion (GRC) is used.

2. INTRODUCCIÓN

Con el avance constante de las tecnologías, los entornos de programación para dispositivos han evolucionado más allá de simples bloques que facilitan la interacción y el conocimiento de distintas plataformas. Ahora, también brindan a los usuarios la posibilidad de crear sus propios algoritmos. De esta manera, el estudiante no solo aprende a manejar los bloques disponibles, sino que adquiere la capacidad de desarrollar nuevas funcionalidades que complementan las actividades ya realizables con GNU Radio.

Esta práctica de laboratorio tiene como finalidad que el estudiante mediante el uso de la herramienta de GNU Radio entienda e implemente 3 tipos de bloques, siendo estos el acumulador, el diferenciador y un tercero el cual sirva para hallar los promedios de tiempo.

3. OBJETIVO GENERAL Y ESPECÍFICOS

1. Objetivo General: Desarrollar competencias que permitan la interacción con la programación de funciones en GNU Radio.

2. Objetivos Específicos:

a. Identificar los aspectos fundamentales en los sistemas de tiempo real y la radio definida por software.
b. Generar funciones a partir de los bloques de implementación de código y evaluar los resultados con otros bloques.
c. Utilizar los bloques implementados para producir una aplicación específica para señales reales.

4. METODOLOGÍA

Para explicar de manera clara y concisa cómo se desarrolló esta práctica de laboratorio es necesario dividir el desarrollo en 2 partes importantes, configuración de GitHub y trabajo en GNU Radio.

- **Configuración del GitHub:** Para dar comienzo a esta práctica, primero fue necesario la creación de ciertas ramas o “branches” las cuales almacenan el trabajo realizado en el laboratorio. Dentro de esta primera rama ,“Práctica 1” se crearon otras tres ramas, correspondientes a cada integrante del equipo en las cuales cada uno realiza sus aportes en el desarrollo de la práctica. Al finalizar la práctica, mediante el comando “merge” se busca unificar cada aporte realizado por cada integrante del grupo en sus respectivas ramas en la rama principal.

- **GNU Radio:** Tras realizar la configuración en GitHub, se procedió a implementar algunos bloques de Python dentro del software GNU Radio, siguiendo las indicaciones del libro guía en la sección 1.2.0.1 proporcionado por el docente. Se incorporaron los bloques de acumulador, diferenciador y promedio de tiempos, cuya funcionalidad fue verificada mediante aplicaciones basadas en los conceptos vistos en clase. Finalmente, utilizando los comandos de GitHub previamente aprendidos, las implementaciones fueron cargadas en la nube dentro de su carpeta correspondiente.

5. ANÁLISIS DE RESULTADOS

1. Bloque Acumulador:

Un bloque acumulador es un sistema discreto que suma de forma progresiva las muestras de entrada, de modo que cada salida corresponde a la suma de la muestra actual con todas las anteriores. Matemáticamente se describe como $y[n] = y[n-1] + x[n]$. Este comportamiento equivale a un integrador en tiempo discreto.

Al momento de implementar el código del acumulador se logró evidenciar que este tenía algunos errores en el, provocando que la señal si se sumara pero de manera indefinida.

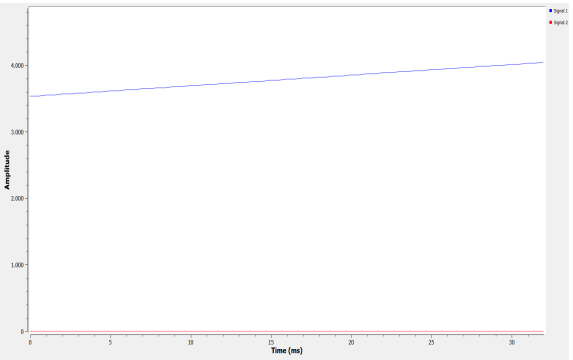

Figura 1. Señal acumulada original.

La solución del problema se obtuvo con ayuda de la inteligencia artificial. Gracias a esta se generó un código auxiliar el cual ayudó a que la señal se centrara y por consiguiente no creciera de manera ilimitada.

```
x_mean = np.mean(x)

for i in range(len(x)):
    self.acum += (x[i] - x_mean)
    y0[i] = self.acum

return len(y0)
```

Figura 2. Código de corrección.

Finalmente, el bloque acumulador logró trabajar de forma correcta entregando una salida que se asemeja a la integral de la señal de entrada.

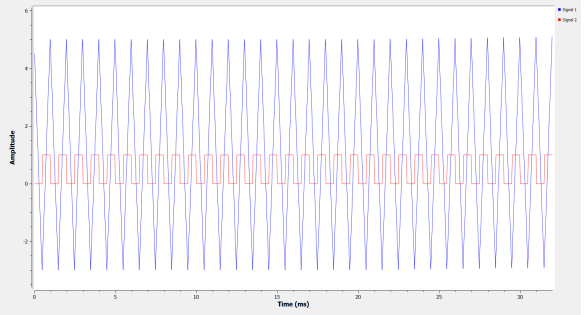

Figura 3. Señal acumulada corregida.

2. Bloque Diferenciador:

Un bloque diferenciador se encarga de calcular la variación instantánea entre muestras consecutivas de la señal de entrada, de modo que cada salida corresponde a la diferencia entre la muestra actual y la muestra inmediatamente anterior. Matemáticamente se puede describir como: $y[n] = x[n] - x[n-1]$.

Este comportamiento equivale a un derivador en tiempo discreto, ya que aproxima la derivada de la señal mediante diferencias finitas. Partiendo del código mostrado en Comunicaciones Digitales basadas en radio definida por software [3]:

```
diff = np.cumsum(x) - self.acum_anterior
self.acum_anterior = diff[N-1]
```

Figura 4. Error en el código original.

El error está en que usar *np.cumsum(x)* no calcula diferencias sino que hace una suma cumulativa.

```
    self.prev = 0.0

def work(self, input_items, output_items):
    x = input_items[0]
    y = output_items[0]

    y[0] = x[0] - self.prev
    y[1:] = x[1:] - x[:-1]

    self.prev = x[-1]
```

Figura 5. Corrección de error en el bloque de python.

La versión corregida almacena la última muestra del bloque anterior para luego calcular la diferencia entre la primera muestra del bloque actual y ese valor previo.

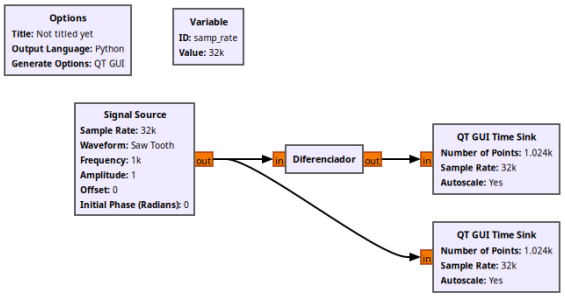

Figura 6. Conexión de bloques para el diferenciador.

Para visualizar de mejor manera el efecto del bloque diferenciador se usaron bloques QT GUI Time Sink tanto para la entrada como para la salida. En el caso de una señal triangular, la salida se asemeja a una señal cuadrada.

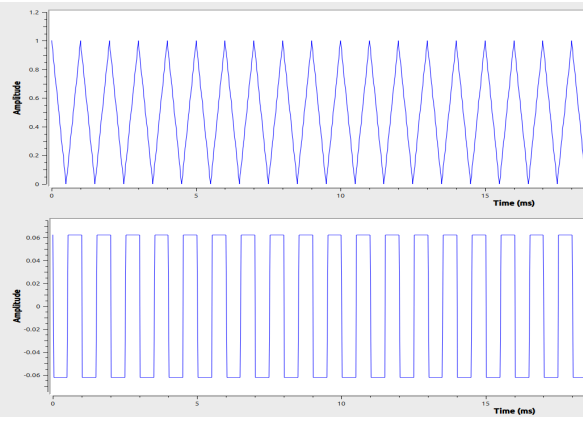

Figura 7. Señal triangular de entrada y señal cuadrada de salida.

3. Bloque promedios de tiempo:

Un bloque de promedios de tiempo es un sistema discreto que calcula parámetros estadísticos de una señal a medida que recibe muestras en el tiempo. A diferencia de un acumulador simple, este bloque no solo suma las muestras, sino que realiza diferentes operaciones matemáticas para obtener la media, la media cuadrática, el valor RMS, la potencia promedio y la desviación estándar.

Matemáticamente, el promedio temporal acumulativo se define como:

$$\bar{x}[n] = \frac{1}{N}\sum_{k=0}^{N-1} x[k]$$

Figura 8. Ecuación de la media discreta de una señal.

Donde N representa el número total de muestras recibidas hasta el instante actual. Este comportamiento implica que el bloque mantiene memoria de las muestras anteriores mediante variables internas acumulativas.

Durante la implementación del bloque se detectó un error en el cálculo de la media cuadrática. El problema se encontraba en la actualización de la variable acumuladora, ya que se estaba almacenando el valor incorrecto al finalizar cada iteración.

El fragmento original presentaba el siguiente error:

```
acumulado1 = self.acum_anterior1 + np.cumsum(x2)
self.acum_anterior1 = acumulado[N-1]   # Error
```

Figura 9. Error en el código.

El problema radica en que se estaba guardando el último valor del acumulado correspondiente a la media simple (acumulado), en lugar del acumulado de la media cuadrática (acumulado1). Esto generaba inconsistencias en los cálculos posteriores.La corrección consistió en actualizar correctamente la variable acumuladora:

```
acumulado1 = self.acum_anterior1 + np.cumsum(x2)
self.acum_anterior1 = acumulado1[N-1]   # Corrección
```

Figura 10. Código de corrección.

Con esta modificación, cada parámetro estadístico mantiene su propio acumulador independiente, garantizando coherencia en los resultados obtenidos.

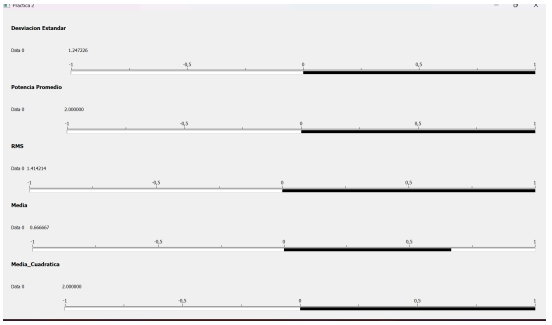

Figura 11. Simulación final en GNU Radio .

6. CONCLUSIONES

- Se verificó que el bloque acumulador actúa como un integrador discreto, sumando progresivamente las muestras de entrada y generando una salida dependiente de su historial.
- Se probaron todas las formas de onda disponibles en el Signal Source como entrada para el diferenciador, aparte del caso mencionado en los resultados, en el caso de un seno la salida es coseno, mientras que para una señal cuadrada la salida es un tren de impulsos, con estos y otras señales más se confirmó el comportamiento

esperado de un sistema derivador discreto.

- Se verificó que el bloque de promedios de tiempo calcula correctamente los parámetros estadísticos de la señal de entrada (media, media cuadrática, RMS, potencia promedio y desviación estándar), utilizando un promedio acumulativo que depende del historial de muestras y cuyos resultados coinciden con los valores teóricos esperados.

- El análisis estadístico de la señal permite caracterizar su comportamiento energético y su distribución en el tiempo. La media indica la presencia o ausencia de componente DC, mientras que el valor RMS y la potencia promedio cuantifican la energía efectiva transmitida. Por otro lado, la desviación estándar proporciona información sobre la dispersión de las muestras respecto al valor medio, permitiendo evaluar el grado de variabilidad de la señal.

7. REFERENCIAS

[1] "Creating your first block." [Online]. Available: https://wiki.gnuradio.org/index.php?title=

Creating_Your_First_Block

[2] "Python block with vectors." [Online]. Available: https://wiki.gnuradio.org/index.php?title=

Python_Block_with_Vectors

[3] "Comunicaciones digitales basadas en radio definida por software." [Online]. Available: https://drive.google.com/file/d/1fd9M4_

bIjwLOajQdkdN9ex2plYRoXomz/view